IIT DELHI x BECHTEL AI/ML PROGRAMME

Handling (Bad) Data

Before any model runs, interrogate your data like an engineer.

Lecture 2

INSPECT

CLEAN

VALIDATE

MODEL

What we covered yesterday

Recap

What AI and ML is

Learning patterns from examples, not hand-written rules

Supervised learning

Give the model features + labels; it learns the mapping

Regression vs Classification

Predict a number (cost) vs predict a class (delayed / not)

The ML workflow

Define → Collect → Prepare → Train → Evaluate → Deploy

Bechtel example

Schedule-delay prediction: crew size, duration, weather risk

What can go wrong

Bad data, overfitting, distribution shift, biased predictions

From ML Models to ML-Ready Data

YESTERDAY		TODAY
How does ML learn from examples?	→	What should ML be allowed to learn from?
Features → Model → Prediction		Raw records → Inspect → Clean → Validate
Train, evaluate, tune in Colab		Apply EPC logic before any algorithm
Clean data, simple workflow		Deliberately messy dataset in Colab

A model is only as reliable as the data pipeline feeding it.

Today's Roadmap

01	**Why bad data is dangerous in EPC** Three real examples from construction projects
02	**Bad data is more than missing values** A taxonomy of EPC data problems
03	**First principles: look before you clean** Inspection habits every engineer should have
04	**Live notebook: synthetic EPC dataset** Level 1 inspection through Level 3 ML diagnosis
05	**NYC infrastructure: real-data reality check** Public data is not automatically clean

> **In EPC projects, AI/ML does not begin with model training. It begins with asking whether the data is meaningful, trustworthy, and decision-ready.**

Today's goal:

Learn to interrogate a project dataset like an engineer before trusting any ML model.

Data cleaning is not clerical work. It is engineering judgement applied to project records.

Example 1 | Wrong units, Wrong productivity

The Story

A work package reports installed pipe length as 3,200 units. The productivity looks outstanding and well above site average.

Project controls flag it for a best-practice case; this crew is performing very well.

Audit reveals: some rows recorded in feet, others in metres.

What the ML model learns

Now imagine we train an ML model on this mixed-unit dataset.

The model does not know physics. It only sees numbers. It may treat these mixed-unit quantities as genuine productivity differences.

Result: confident predictions that are structurally wrong, and hard to spot without domain knowledge.

Key point: The model is not the problem. The data representation is the problem; it is not consistent.

Example 2 | Missing does not mean zero

A safety incident field is blank. What does it mean?

Zero incidents	Report not submitted	Not applicable
Reporting confirmed, nothing happened	Data was never entered	This site type has no incident category

Not yet entered	Data lost in migration
Lag between event and system update	Source system change; field dropped

- If you fill all blanks with zero, you create a false safety picture
- If we train an ML model on this, the model may learn that certain sites, contractors, or project phases are safer than they really are, not because they are actually safer, but because their reports are incomplete

Example 3 | Duplicate is Not Always Duplicate

	Work Pkg	Site	Week	Manhours	Progress
A	WP-1042	Site B	2025-W14	340	62%
B	WP-1042	Site B	2025-W14	**345**	62%

Exact duplicate

Every field is identical.
Pandas **df.duplicated()** catches it.
Safe to drop one row.

Business duplicate

Same project, site, package, week.
But one value differs.
Which row is the authoritative record?
Was one row corrected later?
Did one come from the contractor and one from the planning team?
Do we keep the latest one?

Business duplicates require a domain expert, not just a Pandas command.

Bad Data is More Than Missing Values

- Missing values
- Wrong units
- Inconsistent categories
- Impossible dates
- Business duplicates
- Unclear field definitions
- Human-readable formats
- Biased reporting
- No traceability

Some data is genuinely wrong. Some only looks wrong until a domain expert explains it.

EPC Data-Quality Checklist

Ask these questions before any model touches your data

STRUCTURE

Does each row mean one thing? (one work package, one week, one site)

MISSINGNESS

Does blank mean unknown, zero, not applicable, or not yet reported?

DATES

Are finish dates after start dates? Are formats consistent?

UNITS

Are quantities in mixed units? Are conversions documented?

FIELD MEANING

What does each column mean? What is the unit? Who owns it?

DUPLICATES

Are there exact duplicates? Business duplicates? What is the key?

NUMERIC VALIDITY

Is progress between 0 and 100? Are manhours and costs non-negative?

TRACEABILITY

Is raw data preserved? Is every cleaning step logged?

Today's Live-Demo Roadmap

3.5 to 4 hours | Synthetic EPC dataset + NYC open data

0:00 – 0:20	Framing	Why bad data is dangerous in EPC (this deck)
0:20 – 1:10	Level 1: Inspect	Row meaning, data types, missingness, first questions
1:10 – 1:30	Discussion + Break	What looks suspicious in the dataset?
1:30 – 2:20	Level 2: Clean	Categories, costs, dates, yes/no fields, cleaning log
2:20 – 3:10	Level 3: Diagnose	Naive ML vs domain-informed ML — what do residuals reveal?
3:10 – 3:35	NYC Data	Public infrastructure data: is it ready for ML?
3:35 – 4:00	Takeaways	Checklist + what should Bechtel automate next?

Opening the Notebook

We will now use a synthetic EPC dataset.

The dataset is deliberately messy.

The objective is not to clean it mechanically.

The objective is to ask what each issue means from an EPC and project-control point of view.